The Three Minimal Requirements as Jointly Sufficient: Full Architectural Support for CKS Substrate Hosts

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 4, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **joint-sufficiency** property of the three minimal tool-agnosticism requirements named in §7.1 of the source paper — that any host environment satisfying Requirements 1, 2, and 3 architecturally supports CKS substrates, with no fourth requirement architecturally needed — as a standalone architectural commitment with independent operational content.

Abstract

The CKS pattern's tool-agnosticism commitment names three minimal requirements for any environment that can host a CKS substrate: persistent structured state, human read/write access, and LLM access to substrate content. Separate notes formalize each requirement standalone (A2.24, A2.25, A2.26). This note formalizes a property of the three-requirement *set as a whole*: joint sufficiency. Joint sufficiency states that any host environment meeting all three requirements provides full architectural support for CKS substrates, regardless of which other features the host provides; no fourth requirement is architecturally needed. The note states the four operational components of joint sufficiency, six things the commitment does not claim, four adjacent commitments commonly conflated with it, five downstream commitments it grounds, eight failure modes in which implementations or claims add a fourth requirement implicitly, and an operational test for whether a claim about CKS host requirements respects joint sufficiency. The paired property — that no proper subset of the three requirements is sufficient — is treated separately at A2.28; together the two close the tool-agnosticism decomposition.

1. Why joint sufficiency needs to be formalized as standalone

The parent commitment to tool-agnosticism (A1.05, source §7.1) names three minimal requirements for any environment that can host a CKS substrate. A2.24, A2.25, and A2.26 formalize each requirement standalone. What the three together architecturally produce, and what no fourth requirement is needed to add, is a property of the requirement set as a whole — not a property of any individual requirement — and so requires its own treatment.

The motivating cases are arguments that specific host features are architecturally necessary for CKS: that schema-validation systems must enforce specific schema rules, that hosts must provide particular consistency guarantees, that AI-specific infrastructure (model serving, vector databases, agent runtimes)

must be present, that governance dashboards or specialized governance UIs are required, that fine-grained access-control technology must be specified. Each argument identifies a feature that may be useful in some deployments and treats the feature as if it were architecturally necessary. Joint sufficiency is the architectural commitment that forecloses these arguments: the three requirements are enough, so no fourth requirement is architectural.

Two further motivations make the standalone treatment load-bearing. The strategic prior-art posture: a fixed three-requirement set establishes the architectural floor publicly, so that subsequent claims that any specific host feature is necessary for CKS must overcome a public, dated, attributed commitment to the contrary. And the connection to non-specialist governance (A1.11, source §7.4): commodity-tool sufficiency grounds non-specialist governance, and joint sufficiency makes commodity-tool sufficiency architecturally specific rather than aspirational. Without it, the claim that "a spreadsheet works" remains ambiguous about whether the spreadsheet is a starting point that needs supplementing or a complete architectural host.

2. The joint-sufficiency commitment, defined precisely

In the CKS pattern, the three minimal tool-agnosticism requirements (per A1.05 and source §7.1) are **jointly sufficient** for hosting a CKS substrate if and only if all four of the following hold for any host environment satisfying Requirements 1, 2, and 3.

(a) The host architecturally supports CKS substrates. Substrates hosted on the environment can satisfy the substrate-layer commitments (per A2.08), the cell-layer commitments (per A2.09), and the boundary commitments (per A2.10–A2.12). The host provides the persistence, addressability, direct human access, and LLM read/write access that the architecture requires; substrate content, orchestration rules, and cell behavior can be expressed and operated within the host's affordances.

(b) No additional architectural commitment is required beyond the three requirements. The architecture does not specify a fourth requirement. There is no schema-validation requirement, no specific consistency-model requirement, no AI-runtime requirement, no governance-dashboard requirement, no specific access-control-technology requirement, no observability requirement, no version-control requirement, and no vendor-certification requirement. Hosts meeting the three requirements support full CKS architectural commitments without any further host capability being architecturally specified.

(c) Hosts may provide features beyond the three requirements, but those features are deployment conveniences, not architectural prerequisites. A host with rich query capabilities, sophisticated permission systems, or AI-optimized infrastructure may be more practical, performant, or operationally suitable for specific deployments. The architectural commitment is to the three being sufficient, not to richer hosts being preferred. Optimization for any specific deployment concern is orthogonal to architectural sufficiency.

(d) The architectural support is uniform across hosts meeting the three requirements. A spreadsheet, a relational database, a document store, and a structured-text-file system all support CKS substrates equivalently *at the architectural layer* when each meets the three requirements. Differences between such hosts are deployment-relevant (performance, ergonomics, ecosystem support) but not architecturally relevant. A CKS substrate hosted in any of them is the same architectural object.

The four components together define joint sufficiency. The property is a commitment about the requirement set as a whole; it cannot be decomposed into properties of individual requirements (those are addressed separately by A2.24, A2.25, and A2.26). The phrase that anchors the commitment most concisely is: **no fourth requirement is architecturally needed.**

3. What joint sufficiency does NOT claim

The standalone treatment is not a maximalist treatment. Stating precisely what joint sufficiency does not claim is what keeps the framing from drifting into something stronger than the source paper supports.

It does not claim that all hosts meeting the three requirements are equally good for every deployment. Performance, scalability, ergonomics, and operational characteristics vary across hosts. Architectural sufficiency is a floor, not a ranking; a spreadsheet and a relational database are architecturally equivalent as CKS substrate hosts but not interchangeable for every deployment purpose.

It does not claim that the three requirements are exhaustive in perpetuity. Future architectural extensions of CKS may identify additional requirements that hosts must satisfy to support new capabilities. Joint sufficiency is the claim that the three current requirements are enough for the current architectural commitments; extensions would require their own joint-sufficiency analysis.

It does not claim that meeting the three requirements is easy. Hosts may need to be configured carefully, adapted to provide the specific properties each requirement demands, or supplemented with adapter layers that bridge host features to architectural commitments. Joint sufficiency is about what the host must provide, not about how easily the host provides it.

It does not claim that hosts meeting the three requirements work without deployment effort. The deployment must still design substrate schemas, author orchestration rules, implement cells, configure authority structures, and integrate the system with surrounding organizational context. Joint sufficiency is about host capability, not about deployment completeness.

It does not claim that the three requirements alone make a system useful. A host meeting them supports CKS substrates architecturally; whether the resulting system is useful for any specific coordination purpose depends on the deployment's substrate design, orchestration rules, cell implementations, and integration with organizational context. Architectural sufficiency is necessary for a coherent CKS deployment; it is not sufficient for a useful one.

It does not forbid hosts from being optimized for specific use cases. A host may be optimized for high-throughput workloads, large-scale storage, regulatory compliance, or any other deployment concern, provided it continues to meet the three requirements. A highly-optimized host is no less architecturally sufficient than a minimal one — and no more so.

4. What joint sufficiency is NOT

Four adjacent commitments are commonly conflated with joint sufficiency. Each is a real position in some architectural framing, and each says something different from what joint sufficiency says.

Not best-case host sufficiency. Best-case host sufficiency would mean that the architecturally best host for CKS is one meeting just the three requirements minimally — that adding capabilities beyond the three is in

some way architecturally undesirable. Joint sufficiency does not say that. A host with rich features can be a better fit for a deployment; what makes it architecturally sufficient is the same thing that makes a minimal host architecturally sufficient — meeting the three requirements. The architectural commitment is symmetric across hosts above the floor; deployment fit is not.

Not optimal host sufficiency. Optimal host sufficiency would identify a specific combination of host features beyond the three requirements as the architecturally best configuration. Joint sufficiency makes no claim about optimization. It identifies the architectural floor; configurations above the floor in any combination of features are architecturally sufficient, and the architecture takes no position on which configuration is preferred for any specific purpose.

Not deployment sufficiency. Deployment sufficiency would mean that a deployment as a whole is sufficient when its host meets the three requirements — that nothing further needs to be done. Joint sufficiency is about *host capability*, not about deployment completeness. The deployment must still add substrate schemas, orchestration rules, cell implementations, authority structures, and integration with organizational context; host architectural sufficiency does not eliminate this work.

Not performance sufficiency. Performance sufficiency would mean that any host meeting the three requirements performs adequately for any CKS workload. Joint sufficiency is about architectural support, not performance. A host may meet the three requirements but perform poorly for a specific workload — a flat-file substrate may be architecturally sufficient but unusable for high-frequency reads. Performance is a deployment property the architecture leaves to deployment design.

5. Why joint sufficiency is load-bearing for downstream commitments

Joint sufficiency grounds five downstream commitments in the CKS architecture.

The tool-agnosticism commitment (A1.05). The parent commitment that “any host meeting the three requirements works” is architecturally specific only because joint sufficiency forecloses fourth-requirement claims. Without it, the parent commitment would remain procedurally vague and would still permit unspecified host expectations to emerge as architectural over time. Joint sufficiency is what makes the parent commitment defensible.

The non-specialist governance commitment (A1.11, source §7.4). Non-specialist governance commits the architecture to commodity-tool accessibility for the three rights humans hold over substrate content and orchestration rules. The commitment depends on commodity tools being architecturally sufficient hosts. Joint sufficiency is what supplies that sufficiency: a spreadsheet meets Requirements 1, 2, and 3, and joint sufficiency states that meeting those three is architecturally complete. No specialist tool, governance platform, or AI runtime is architecturally required, and the source paper's commitment to non-specialist governance stands or falls on that statement being defensible.

The architectural-property qualifier on governance (A2.06). A2.06 formalizes that governance in CKS is an architectural property, not a procedural promise that depends on a particular deployment. Joint sufficiency contributes to that qualifier specifically at the host-requirements layer: the host requirements *are* architectural — three named requirements, no fourth — with no procedural fourth requirement that deployments must discover or satisfy by convention.

The migration-safety property. A CKS substrate can be migrated between any two hosts that meet the three requirements without architectural commitment loss. Migration correctness depends on the determinism contract (per A1.10); migration *permission* — the architectural permission to migrate at all between any qualifying hosts — comes from joint sufficiency. Because all hosts meeting the three requirements are architecturally equivalent for substrate purposes, migration between them preserves the architecture. Without joint sufficiency, hosts could differ on unspecified fourth requirements, and migration would not be architecturally safe.

The strategic prior-art posture. The architectural commitment to a fixed three-requirement set is publicly documented as architecturally complete, with no fourth requirement claimed. Joint sufficiency formalizes the prior-art territory the tool-agnosticism decomposition occupies: subsequent claims that any specific host feature is architecturally necessary for CKS must overcome the public commitment to the contrary.

These are not new commitments. Each is grounded elsewhere in the source paper or in a prior derivation note; what joint sufficiency does is make each of them defensible in turn.

6. Failure modes that violate joint sufficiency

Each of the following anti-patterns is observable in the 2024–2026 governance-first AI infrastructure landscape. Each looks like a reasonable host expectation in some deployment context, and each implicitly adds a fourth requirement that joint sufficiency forecloses.

(a) Implicit schema-validation requirement. Hosts are claimed to need specific schema-rule enforcement to support CKS substrates. The substrate's schema is a deployment decision (a property of the cell, not of the host); host enforcement of schemas is a deployment convenience. Treating schema validation as architecturally required adds a fourth requirement.

(b) Implicit consistency-model requirement. Hosts are claimed to need specific consistency guarantees — strong consistency, ACID transactions, linearizability — to support CKS substrates. The substrate's consistency needs are deployment-relevant; the host's specific consistency model is not architecturally specified. Treating consistency-model strength as architecturally required adds a fourth requirement.

(c) Implicit AI-runtime requirement. Hosts are claimed to need AI-specific infrastructure — model serving, vector databases, agent runtimes — to support CKS substrates. Requirement 3 specifies that LLM access works through whatever read/write mechanism the host provides; AI-specific infrastructure is a deployment convenience, not architectural. Treating AI infrastructure as architecturally required adds a fourth requirement, and is the most common architectural-overreach pattern in vendor-positioned governance-first claims.

(d) Implicit governance-dashboard requirement. Hosts are claimed to need governance dashboards or specialized governance UIs to support CKS substrates. Requirement 2 specifies direct human access through standard read/write operations; dashboards are deployment conveniences, not architectural. Treating governance UIs as architecturally required adds a fourth requirement and undermines the non-specialist governance commitment that depends on commodity-tool sufficiency.

(e) Implicit access-control-system requirement. Hosts are claimed to need fine-grained access-control systems with specific features (role-based access control, attribute-based access control, policy engines) to

support CKS substrates. The architecture supports authority partitioning (per A1.01), but specific access-control technology is a deployment choice. Treating particular access-control technology as architecturally required adds a fourth requirement.

(f) Implicit observability requirement. Hosts are claimed to need audit logging, observability, or monitoring infrastructure to support CKS substrates. Path retraceability (per A1.07) is satisfied through substrate provenance metadata; external observability infrastructure is a deployment concern. Treating observability infrastructure as architecturally required adds a fourth requirement.

(g) Implicit version-control requirement. Hosts are claimed to need version control over substrate content to support CKS substrates. Substrate persistence (Requirement 1) covers durability; version control is a deployment convenience for tracking change history beyond what provenance metadata records. Treating version control as architecturally required adds a fourth requirement.

(h) Implicit "vetted-host" requirement. Only specific hosts (vendor-blessed, certified, qualified) are claimed to support CKS substrates, even when other hosts meet the three requirements. The architectural commitment is to any host meeting the three requirements; a vetted-host requirement adds a fourth requirement that is neither architecturally specified nor publicly defined, and effectively binds CKS to a specific vendor.

Each of (a)–(h) names a feature that may genuinely benefit a deployment. Joint sufficiency is what makes visible that each is a deployment concern, not an architectural requirement; the fourth-requirement test in each case is whether the claim still holds when the feature is absent and the three requirements are met.

7. Operational test

A claim about CKS host requirements respects the joint-sufficiency commitment if and only if all of the following are true.

1. The claim does not assert any fourth architectural requirement beyond Requirements 1, 2, and 3 (per A2.24, A2.25, A2.26). Specific host features beyond the three may be discussed as deployment conveniences, but they are not asserted as architectural prerequisites.

2. The claim acknowledges that hosts meeting the three requirements support CKS substrates architecturally, regardless of whether the hosts provide additional features.

3. The claim distinguishes architectural support (governed by joint sufficiency) from deployment suitability (which may favor hosts with richer features for specific purposes).

4. The claim does not require specific host technology, vendor, or feature set as a precondition for CKS architectural coherence.

5. The claim is consistent with the migration property — substrates can be moved between any hosts meeting the three requirements without architectural commitment loss.

A claim that fails any of (1)–(5) violates joint sufficiency in at least one direction; it adds a fourth requirement either explicitly or implicitly, and the architectural commitment to the three-requirement set as architecturally complete does not hold under such a claim.

8. Why naming joint sufficiency as standalone matters

Implementations under pressure to advocate for specific host technologies, vendor-specific platforms, or AI-specific infrastructure consistently drift toward claiming that their preferred features are architecturally necessary. The drift is steady because each feature offers genuine benefits, and arguing that the feature is "needed" for CKS is a way of advocating for it within architectural rather than deployment vocabulary. Implementations that drift away from joint sufficiency produce a CKS landscape where the architectural floor is unclear, vendor lock-in becomes architectural rather than practical, and the architecture's claim to host-agnosticism becomes ambiguous.

Joint sufficiency is the architectural commitment that prevents this drift by establishing the floor as fixed at three requirements. The standalone formalization — with the four components in §2, the limitations in §3, the four adjacent-pattern distinctions in §4, the five load-bearing connections in §5, and the eight failure modes in §6 — gives downstream readers a precise specification of what the three-requirement set as a whole architecturally produces, and of what is foreclosed by the choice to fix the requirement set at three. **No fourth requirement is architecturally needed**, and any claim that adds one departs from the architecture even when the added requirement is independently desirable.

The paired property — that no proper subset of the three requirements is sufficient — is treated separately at A2.28. Together with this note, A2.28 specifies the architectural relationship structure of the three-requirement set: the three together are enough (joint sufficiency), and no fewer than three are enough (individual necessity). The two properties close the tool-agnosticism decomposition. Subsequent work that adopts, extends, composes with adjacent patterns, or argues against the CKS tool-agnosticism commitment should use both properties in the senses formalized here. Subsequent work that uses either property differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Three Minimal Requirements as Jointly Sufficient: Full Architectural Support for CKS Substrate Hosts.* May 4, 2026. ORCID: 0009-0004-8065-3235.